Laporan Tugas Besar 1

IF2211 Strategi Algoritma

Pemanfaatan Algoritma *Greedy* dalam pembuatan bot permainan Battlecode 2025

Semester II Tahun 2025/2026

v1.0

DEADLINE: MINGGU, 15 MARET 2026, 23:59 WIB

Disusun oleh Kelompok 2 (Nayakuy):
13524053 - Muhammad Haris Putra Sulastianto
13524071 - Kalyca Nathania Benedicta Manullang
13524091 - Vara Azzara Ramli Pulukadang

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

1 Deskripsi Tugas

1.1 Latar Belakang

Battlecode adalah permainan strategi waktu nyata di mana pemain menulis kode untuk mengendalikan robot secara otonom. Robot-robot tersebut harus dikelola secara strategis untuk bekerja sama dan mengalahkan tim lawan. Dalam permainan ini, peserta memanfaatkan berbagai konsep seperti kecerdasan buatan, penelusuran jalur (pathfinding), algoritma terdistribusi, serta komunikasi antar unit.

Namun, setiap robot memiliki keterbatasan sumber daya komputasi pada setiap giliran. Oleh karena itu, pemain harus merancang strategi yang efisien, melakukan optimasi, dan menentukan kompromi yang tepat.

Pada Tugas Besar 1 mata kuliah Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang akan dipertandingkan dengan bot dari kelompok lain. Bot tersebut harus dirancang menggunakan pendekatan algoritma greedy.

1.2 Komponen Permainan

1.2.1 Objectives

Tujuan utama permainan adalah mewarnai peta. Tim pertama yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai akan memenangkan permainan. Tim juga dapat menang dengan cara menghancurkan seluruh robot dan menara milik tim lawan.

1.2.2 Aturan Tiebreaker

Jika setelah 2000 ronde tidak ada tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai, maka permainan berakhir dan pemenang ditentukan berdasarkan urutan berikut:

- Luas area yang sudah diwarnai di peta
- Jumlah menara sekutu yang masih hidup
- Total jumlah uang (chips)
- Jumlah total cat (paint) yang dimiliki semua robot dan menara
- Jumlah robot sekutu yang masih hidup
- Jika masih seri, tim dipilih secara acak

1.2.3 Peta

Setiap permainan dimainkan pada peta berbentuk grid dua dimensi dengan ukuran antara 20×20 hingga 60×60 petak.

1.2.4 Jarak Pandang dan Pergerakan

Robot dapat melihat fitur peta dan robot lain dalam jarak $\sqrt{20}$.

1.2.5 Tower dan Ruins

Menara berfungsi sebagai zona spawn tim dan dapat menyerang unit musuh.

1.2.6 Pola Sumber Daya Khusus

Pola tertentu pada peta dapat memberikan bonus sumber daya yang disebut Special Resource Pattern (SRP).

1.2.7 Cat

Cat (paint) merupakan sumber daya utama yang digunakan untuk mengklaim wilayah.

1.2.8 Chips

Chips digunakan untuk membangun robot dan menara.

1.2.9 Robot

Semua robot dapat bergerak, melakukan sensing, dan berkomunikasi dengan robot lain. Permainan berjalan secara turn-based dan dibagi menjadi ronde.

1.2.10 Jenis Robot

Soldier

- HP: 250
- Kapasitas cat maksimum: 200
- Biaya produksi: 200 cat + 250 chips

Splasher

- HP: 150
- Kapasitas cat maksimum: 300
- Biaya produksi: 300 cat + 400 chips

Mopper

- HP: 50
- Kapasitas cat maksimum: 100
- Biaya produksi: 100 cat + 300 chips

1.2.11 Menara

Menara dibangun di atas reruntuhan setelah pola cat tertentu selesai dibuat.

1.2.12 Serangan

Menara memiliki dua jenis serangan: single-target dan area-of-effect.

1.2.13 Komunikasi

Robot dan menara dapat berkomunikasi menggunakan pesan maksimal 4 byte.

2 Landasan Teori

2.1 Algoritma Greedy

Algoritma greedy merupakan metode sederhana dan populer yang digunakan untuk menyelesaikan persoalan optimasi, yaitu persoalan yang bertujuan mencari solusi terbaik dari sekian banyak kemungkinan solusi. Dalam algoritma greedy, solusi dibangun secara bertahap (step by step). Pada setiap langkah, algoritma mengambil keputusan terbaik yang dapat diperoleh saat itu (optimum lokal) tanpa mempertimbangkan konsekuensi ke depan, dengan harapan bahwa serangkaian keputusan lokal tersebut akan mengarah pada solusi optimum global.

Prinsip greedy: *"take what you can get now!"*

Algoritma greedy memiliki sifat-sifat berikut:

- **Greedy choice property**: Solusi optimal dapat dibangun dengan memilih pilihan lokal terbaik terlebih dahulu.
- **Optimal substructure**: Solusi optimal dari persoalan mengandung solusi optimal dari sub-persoalannya.

Komponen-komponen utama dalam algoritma greedy meliputi:

1. **Himpunan kandidat** (C): berisi elemen-elemen yang dapat dipilih pada setiap langkah.
2. **Himpunan solusi** (S): kumpulan kandidat yang telah dipilih dan membentuk solusi.
3. **Fungsi solusi**: menentukan apakah himpunan solusi S sudah lengkap dan memenuhi kriteria.
4. **Fungsi seleksi**: memilih kandidat terbaik dari himpunan C berdasarkan suatu strategi greedy (heuristik).
5. **Fungsi kelayakan**: memeriksa apakah kandidat yang dipilih dapat dimasukkan ke dalam himpunan solusi tanpa melanggar batasan.
6. **Fungsi objektif**: fungsi yang dioptimalkan, baik maksimasi maupun minimasi.

Contoh klasik penerapan algoritma greedy adalah persoalan penukaran uang dengan koin, pemilihan aktivitas (activity selection problem), fractional knapsack, pohon merentang minimum (Prim dan Kruskal), lintasan terpendek (Dijkstra), serta penjadwalan job dengan tenggat waktu. Pada beberapa persoalan seperti fractional knapsack dan penjadwalan dengan LPT, algoritma greedy terbukti memberikan solusi optimal,

namun pada banyak kasus lain (misalnya integer knapsack, TSP, atau penukaran uang dengan pecahan tidak kanonik) algoritma greedy hanya menghasilkan solusi sub-optimum atau pseudo-optimum. Oleh karena itu, pemilihan fungsi seleksi yang tepat sangat menentukan keberhasilan algoritma greedy.

2.2 Penerapan Algoritma Greedy dalam Bot Battlecode

2.2.1 Sekilas tentang Battlecode 2025

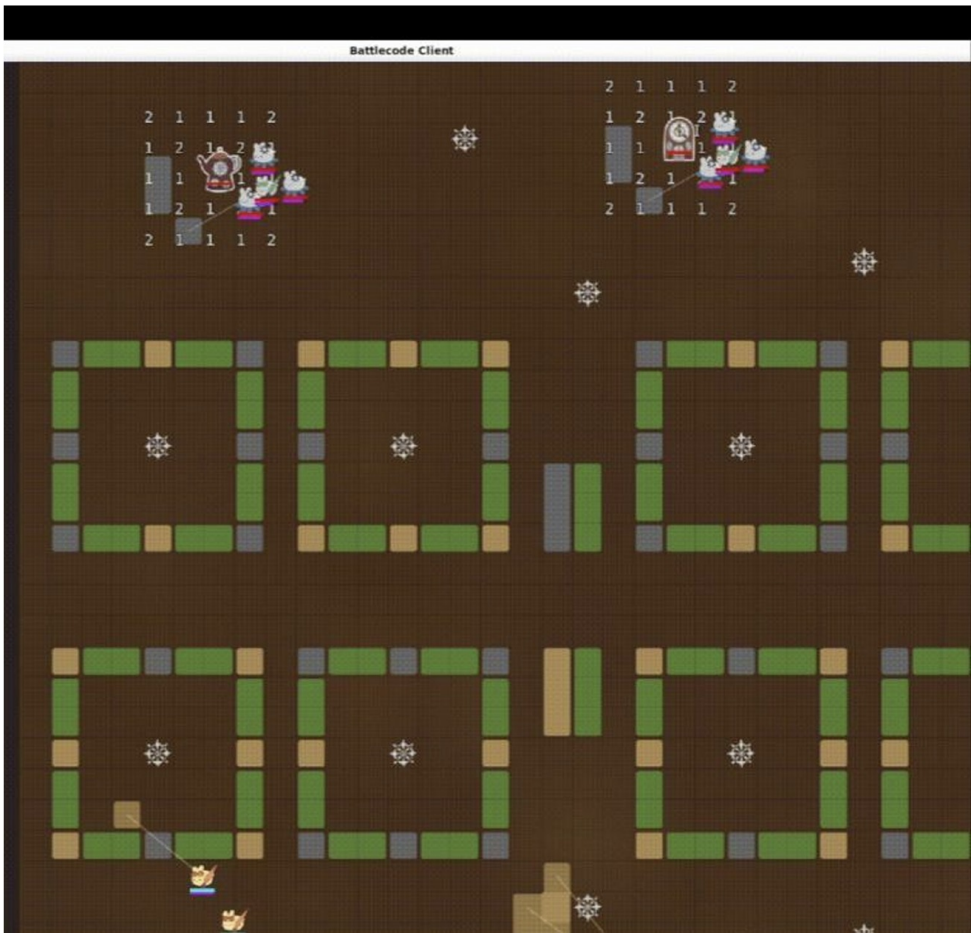


Figure 1: Tampilan Permainan Battlecode 2025

Battlecode adalah permainan strategi waktu nyata berbasis giliran (turn-based) yang dimainkan pada peta grid berukuran 20×20 hingga 60×60. Setiap tim mengendalikan sejumlah robot otonom yang dikendalikan oleh kode yang ditulis dalam bahasa Java. Tujuan utama permainan adalah mewarnai lebih dari 70% petak yang dapat diwarnai, atau menghancurkan seluruh robot dan menara lawan.

Terdapat tiga jenis robot: **Soldier** (penyerang satu petak, mewarnai, merusak menara), **Splasher** (serangan area, mewarnai luas), dan **Mopper** (menghapus cat musuh, mentransfer cat, dan serangan sapu). Setiap robot memiliki batasan cat (paint) dan chips, serta memiliki cooldown untuk bergerak dan melakukan aksi. Robot juga dibatasi oleh jumlah bytecode per giliran (17.500 untuk robot, 20.000 untuk menara) yang membatasi kompleksitas komputasi.

Komunikasi antar robot dilakukan melalui pesan 4 byte atau penanda (marker) di peta, tetapi hanya dapat dilakukan jika terhubung oleh jalur cat sekutu.

2.2.2 Cara Kerja Bot

Setiap robot menjalankan salinan kode yang sama secara independen. Pada setiap ronde, semua robot mendapat giliran untuk menjalankan kodenya secara berurutan. Kode yang ditulis harus efisien karena dibatasi bytecode. Robot dapat melakukan berbagai aksi seperti bergerak, menyerang/mewarnai, menandai, mentransfer cat, menarik cat dari menara, mengirim pesan, atau menghancurkan diri. Setiap aksi memiliki persyaratan cooldown dan biaya sumber daya.

Untuk menjalankan bot, tim mengembangkan kode dalam folder `src` menggunakan struktur yang telah ditentukan. Kemudian, dengan bantuan Gradle dan client yang disediakan, bot dapat dijalankan dan dipertandingkan. Langkah-langkahnya meliputi:

1. Meng-clone repository yang disediakan asisten.
2. Membangun program dengan perintah `./gradlew build`.
3. Menjalankan client dan memilih direktori root.
4. Memulai pertandingan antar bot.

2.2.3 Implementasi Algoritma Greedy dalam Bot

Dalam konteks pembuatan bot Battlecode, algoritma greedy digunakan untuk memilih aksi terbaik yang akan dilakukan oleh setiap robot pada setiap gilirannya. Karena robot tidak memiliki pandangan global dan hanya dapat mengamati lingkungan sekitar, keputusan harus diambil berdasarkan informasi lokal dengan harapan memberikan kontribusi maksimal terhadap tujuan tim.

Pemetaan elemen greedy ke dalam permainan:

- **Himpunan kandidat**: Berupa semua kemungkinan aksi yang dapat dilakukan robot pada giliran tersebut, misalnya: bergerak ke suatu arah, menyerang petak tertentu, membangun menara di reruntuhan terdekat, mentransfer cat, dll. Kandidat juga dapat berupa target (misalnya musuh dengan HP terendah, area yang belum diwarnai, reruntuhan terdekat).
- **Fungsi seleksi**: Heuristik yang digunakan untuk memilih aksi terbaik. Beberapa contoh heuristik yang dapat diterapkan:
  - *Maximize area gain*: memilih aksi yang diperkirakan akan mewarnai area terluas pada giliran tersebut (misalnya dengan Splasher).
  - *Weakest enemy first*: memilih target musuh dengan HP terendah untuk dieliminasi.
  - *Economic greed*: memprioritaskan pembangunan menara uang untuk mempercepat produksi chips.
  - *Closest frontier*: bergerak menuju batas wilayah (frontier) antara area sekutu dan netral/musuh untuk memperluas kontrol.

- **Fungsi kelayakan**: Memastikan bahwa aksi yang dipilih dapat dilakukan pada kondisi saat itu. Misalnya: apakah robot memiliki cukup cat untuk menyerang, apakah cooldown aksi sudah siap, apakah petak tujuan tidak terhalang tembok atau robot lain, apakah robot dalam jangkauan untuk menyerang.
- **Fungsi objektif**: Tujuan akhir yang ingin dicapai tim, yaitu memenangkan permainan dengan menguasai area seluas mungkin atau menghancurkan lawan. Dalam greedy, fungsi objektif diterjemahkan menjadi memaksimalkan kontribusi lokal terhadap tujuan global.

3 Aplikasi Strategi Greedy

3.1 Alternatif Strategi 1: Money Tower Spam

Strategi *Money Tower Spam* berfokus pada pembangunan menara uang secara agresif di awal permainan untuk mengamankan pasokan *chips* yang besar. Dengan ekonomi yang kuat, tim dapat memproduksi robot dalam jumlah besar dan mendominasi pertengahan hingga akhir permainan.

3.1.1 Strategi Ekonomi Dinamis (Keputusan Tower)

Alih-alih menggunakan batasan ronde yang kaku, tower menggunakan pendekatan greedy terhadap sumber daya tim.

- **Himpunan Kandidat**: Memilih antara membangun *Money Tower* atau *Paint Tower*.
- **Himpunan Solusi**: Kumpulan keputusan pembangunan tower yang diambil selama permainan.
- **Fungsi Solusi**: Solusi dianggap lengkap ketika tidak ada lagi ruin yang tersedia untuk membangun tower, atau ketika permainan berakhir.
- **Fungsi Seleksi**: Mengecek jumlah *chips* dan *paint* yang dimiliki tim saat ini.
- **Fungsi Kelayakan**: Jika *chips* tim ≥ 2500, sistem menganggap pasokan uang sudah cukup stabil (kenyang) dan akan memprioritaskan pembangunan *Paint Tower* untuk menjamin ketersediaan amunisi. Jika *chips* < 2500, maka *Money Tower* menjadi prioritas utama.
- **Fungsi Objektif**: Memastikan ketersediaan cat (amunisi) yang maksimal untuk pasukan setelah modal finansial stabil sehingga tidak terjadi kelangkaan cat di tengah ekspansi.

3.1.2 Seleksi Target Serangan Area (Splasher)

Unit *Splasher* menggunakan prinsip *greedy local optimum* untuk memaksimalkan dampak setiap serangan (biaya 50 cat).

- **Himpunan Kandidat**: Semua lokasi yang berada dalam jangkauan serangan (*actionRadiusSquared*) robot.

- **Himpunan Solusi**: Satu lokasi target yang dipilih untuk diserang pada giliran tersebut.
- **Fungsi Solusi**: Solusi ditemukan ketika robot memutuskan untuk menyerang atau memilih untuk tidak menyerang (bergerak).
- **Fungsi Seleksi**: Melakukan scanning pada semua lokasi yang berada dalam jangkauan serangan (*actionRadiusSquared*) robot.
- **Fungsi Objektif (Scoring)**: Menghitung potensi keuntungan dari setiap lokasi target dengan sistem skor:
  - Ubin musuh: skor +2 (karena menghambat lawan).
  - Ubin kosong/netral: skor +1 (memperluas wilayah).
- **Fungsi Kelayakan**: Serangan hanya dilakukan jika total skor ≥ 4 (atau setara dengan 4 ubin kosong). Jika ubin yang bisa dicat kurang dari 4, robot akan memilih untuk bergerak (menyimpan amunisi untuk target yang lebih menguntungkan nanti).
- **Fungsi Objektif**: Memaksimalkan jumlah ubin yang diubah per serangan sehingga efisiensi cat dan *cooldown* menjadi optimal.

3.1.3 Logistik Pengisian Ulang (Mopper)

Mopper bertindak sebagai pendukung logistik dengan logika greedy untuk menjaga momentum serangan.

- **Himpunan Kandidat**: Unit *Splasher* atau *Soldier* di sekitar yang membutuhkan cat.
- **Himpunan Solusi**: Satu unit yang dipilih untuk menerima transfer cat.
- **Fungsi Solusi**: Solusi tercapai ketika *Mopper* berhasil mentransfer cat atau memutuskan untuk tidak mentransfer.
- **Fungsi Seleksi**: Mencari unit *Splasher* dengan persentase cat paling rendah (paling kritis).
- **Fungsi Kelayakan**: Transfer cat hanya dilakukan jika stok cat *Mopper* sendiri masih aman, yaitu > 30 unit. Hal ini mencegah *Mopper* kehabisan cat dan ikut tidak produktif.
- **Fungsi Objektif**: Meminimalkan waktu *downtime* unit penyerang (*Splasher*). Dengan mencegat mereka yang mundur untuk mengisi ulang, *Mopper* mempercepat kembalinya unit penyerang ke garis depan.

3.1.4 Navigasi dan Eksplorasi (Pergerakan)

Pergerakan robot menggunakan *greedy directional selection* untuk menghindari kerumunan dan memperluas eksplorasi.

- **Himpunan Kandidat**: 8 arah mata angin yang mungkin untuk bergerak.

- **Himpunan Solusi**: Satu arah yang dipilih untuk bergerak.
- **Fungsi Solusi**: Solusi ditemukan ketika robot memutuskan untuk bergerak ke suatu arah atau diam.
- **Fungsi Seleksi (Anti-Gerombolan)**: Menghitung vektor gaya menjauh dari robot kawan terdekat. Arah yang menjauhi kerumunan (*repulsion force*) diberi bobot lebih tinggi.
- **Fungsi Kelayakan**: Tujuan pergerakan harus merupakan petak kosong yang dapat dilalui (bukan tembok, bukan robot lain).
- **Fungsi Objektif**: Memastikan robot menyebar secara merata untuk membuka *fog of war* dan menemukan reruntuhan (*ruins*) baru secepat mungkin. Hal ini mendukung strategi utama dengan mempercepat akses ke lokasi pembangunan tower.

3.1.5 Prioritas Produksi Unit (Tower Spawning)

Tower mengambil keputusan produksi unit secara instan berdasarkan prioritas kebutuhan.

- **Himpunan Kandidat**: Memproduksi *Soldier*, *Splasher*, *Mopper*, atau melakukan *upgrade* tower.
- **Himpunan Solusi**: Satu jenis unit yang diproduksi atau keputusan untuk melakukan *upgrade*.
- **Fungsi Solusi**: Solusi tercapai ketika tower berhasil memproduksi unit atau melakukan upgrade.
- **Fungsi Seleksi** (berdasarkan prioritas):
  1. **Prioritas 1 (Ekspansi)**: Jika ada *ruin* kosong yang terdeteksi, produksi *Soldier* untuk segera membangun tower baru.
  2. **Prioritas 2 (Dominasi)**: Jika *chips* tim melimpah (> 3000), produksi *Splasher* untuk memperluas wilayah secara masif.
  3. **Prioritas 3 (Pertahanan)**: Jika ada unit musuh dalam radius ancaman, produksi *Mopper* untuk dukungan dan pembersihan cat musuh.
- **Fungsi Kelayakan**: Tim harus memiliki sumber daya yang cukup (cat dan chips) untuk memproduksi unit yang dipilih.
- **Fungsi Objektif**: Menjaga keseimbangan antara ekspansi, dominasi, dan pertahanan dengan memanfaatkan kelebihan sumber daya yang ada.
- **Optimasi**: Jika semua aksi tidak terpenuhi, gunakan uang untuk Upgrade Tower guna meningkatkan statistik secara permanen.

3.2 Alternatif Strategi 2: Weakest Enemy First

Strategi *Weakest Enemy First* fokus pada agresi dan serangan langsung ke tower musuh dengan memanfaatkan simetri peta. Ketika tower musuh hancur, sumber daya untuk menciptakan unit-unit baru akan berkurang secara signifikan. Dengan begitu, game akan dimenangkan dengan menghancurkan setiap tower dan unit bot musuh. Unit robot akan mewarnai tile dalam perjalanan ke tower musuh dengan tujuan untuk mengurangi penalti cat sehingga bisa menyerang dengan lebih optimal, bukan untuk tujuan ekspansi wilayah.

3.2.1 Keputusan Produksi Tower

Tower menggunakan pendekatan greedy yang hanya menjamin siklus produksi unit tidak macet.

- **Himpunan Kandidat**: Produksi Soldier, Mopper, Splasher, atau Self-Upgrade Tower.
- **Himpunan Solusi**: Satu jenis aksi pada giliran tersebut.
- **Fungsi Solusi**: Solusi tercapai jika tower berhasil spawn unit, upgrade, atau tidak melakukan apa-apa karena kekurangan sumber daya.
- **Fungsi Seleksi**: Mengecek ketersediaan chips dan membaca indeks antrean produksi dengan perbandingan Soldier:Mopper:Splasher adalah 2:1:1.
- **Fungsi Kelayakan**:
  1. Jika chips > 3000 layak self-upgrade
  2. Jika chips ≥ 500 layak produksi Splasher
  3. Jika chips < 500 layak produksi Soldier dan Mopper
- **Fungsi Objektif**: Aliran produksi tidak berhenti dan mempertahakan rasio unit sekutu secara konsisten.

3.2.2 Prioritas Soldier

Bertujuan menyerang sambil mengecat tile untuk dirinya sendiri.

- **Himpunan Kandidat**: Tower terdekat, ruin terdekat, dan tile kosong terdekat.
- **Himpunan Solusi**: Menyerang tower musuh, membangun tower di atas ruin, atau mengecat tile untuk dirinya sendiri.
- **Fungsi Solusi**: Solusi tercapai ketika berhasil menyerang tower musuh atau membangun tower sekutu.
- **Fungsi Seleksi**:
  1. Cari tower musuh dengan HP terendah.
  2. Jika tidak ada tower musuh, cari ruin terdekat dan cat pola tower sekutu.
  3. Jika tidak ada ruin, cat tile diri sendiri.

- **Fungsi Kelayakan**: Action ready dan berada dalam jangkauan serangan
- **Fungsi Objektif**: Menyerang tower musuh sambil membangun ekonomi sendiri apabila tidak terlihat.

3.2.3 Prioritas Splasher

Splasher menggunakan serangan areanya pada keadaan tertentu.

- **Himpunan Kandidat**: Tower terlemah atau tile yang berada dalam radius penyerangan maksimal
- **Himpunan Solusi**: Titik koordinat target untuk diserang
- **Fungsi Solusi**: Ditemukan ketika unit memutuskan untuk menyerang target
- **Fungsi Seleksi**:
  1. Cari tower musuh
  2. Jika tidak ada tower musuh, menimpa cat musuh terdekat
  3. Mewarnai tile kosong
- **Fungsi Kelayakan**: Serangan tile hanya dieksekusi ketika total akumulasi skor pada titik tersebut ≥ 4. Jika tidak, tembakan ditahan
- **Fungsi Objektif**: Memaksimalkan perebutan wilayah musuh walaupun cooldown Splasher besar.

3.2.4 Prioritas Mopper

Mopper bukan berperan sebagai logistik, melainkan penghisap cat musuh atau pemberi cat sekutu di depan.

- **Himpunan Kandidat**: Unit musuh dan unit sekutu terdekat.
- **Himpunan Solusi**: Menghisap cat musuh atau menolong unit sekutu.
- **Fungsi Solusi**: Menghisap cat musuh atau menolong unit sekutu.
- **Fungsi Seleksi**:
  1. Menghisap cat musuh dengan rasio paling kecil.
  2. Memberi cat kepada unit sekutu yang kritis.
- **Fungsi Kelayakan**: Mopper hanya memberi cat jika cat unitnya ≥ 50.
- **Fungsi Objektif**: Unit Mopper yang bergerak mandiri, menyerap sumber daya musuh, dan mencegah antrean di dekat Paint Tower sekutu.

3.2.5 Navigasi

Pergerakan unit dengan *greedy directional selection.*

- **Himpunan Kandidat**: 8 arah mata angin (termasuk arah mundur) dan 3 kemungkinan lokasi markas musuh berdasarkan rotasi simetri peta.
- **Himpunan Solusi**: Satu tebakan lokasi markas musuh yang aktif, dan satu langkah arah pergerakan.
- **Fungsi Solusi**: Unit sekutu beralih simetri atau berhasil berpindah satu petak.
- **Fungsi Seleksi**:
  1. Jika titik target bukan tower musuh, maka unit mengganti arah simetri
  2. Saat bergerak, unit punya prioritas untuk menginjak tile sekutu, tile kosong, baru tile musuh.
- **Fungsi Kelayakan**: Tile yang bisa dilewati. Jika arahnya buntu, unit akan mundur.
- **Fungsi Objektif**: Mengurangi kematian sia-sia karena berjalan di tile musuh dan memangkas waktu eksplorasi ke tower musuh.

3.2.6 Prioritas Menyerang

Soldier, Mopper, dan Splasher akan memprioritaskan untuk menyerang tower atau unit lawan terlemah terlebih dahulu.

- **Himpunan Kandidat**: seluruh unit musuh yang berada di dalam jangkauan serangan.
- **Himpunan Solusi**: Satu unit musuh yang dijadikan fokus serangan.
- **Fungsi Solusi**: Unit sekutu berhasil memberikan serangan.
- **Fungsi Seleksi**: Mencari HP terendah di antara musuh dalam jangkauan serangan. Mopper cenderung menyerang tower ketimbang unit robot musuh.
- **Fungsi Kelayakan**: Target serangan valid jika berada dalam jangkauan serangan.
- **Fungsi Objektif**: Mengurangi sumber daya musuh dengan mengeliminasi unit musuh yang paling dekat dengan kematian yang paling cepat dan efisien saat itu.

3.3 Alternatif Strategi 3: Frontier Expansion Greedy

Strategi *Frontier Expansion Greedy* berfokus pada perluasan wilayah secara agresif dengan memprioritaskan area terdepan (*frontier*) yang belum dikuasai. Setiap unit mengambil keputusan berdasarkan skor ekspansi yang dihitung secara lokal dengan tujuan memperluas area kendali secepat mungkin sambil tetap merespons ancaman musuh.

3.3.1 Keputusan Produksi Tower (Tower Spawning)

Tower memproduksi unit secara adaptif berdasarkan kebutuhan medan perang dengan heuristik sederhana.

- **Himpunan Kandidat**: Memproduksi *Soldier*, *Splasher*, atau *Mopper*.
- **Himpunan Solusi**: Satu jenis unit yang dipilih untuk diproduksi pada giliran tersebut.
- **Fungsi Solusi**: Solusi tercapai ketika tower berhasil memproduksi unit atau memutuskan untuk tidak memproduksi karena keterbatasan sumber daya.
- **Fungsi Seleksi**:
  1. Jika jumlah musuh di sekitar ≥ 3, produksi *Splasher* untuk memberikan tekanan area.
  2. Jika terdapat banyak cat musuh di sekitar (dideteksi oleh *Helper.isMopperNeeded()*), produksi *Mopper* untuk pembersihan.
  3. Selain itu, produksi *Soldier* untuk ekspansi wilayah.
- **Fungsi Kelayakan**: *Chips* tim > cadangan aman (kecuali di awal permainan) dan *paint* mencukupi untuk memproduksi unit yang dipilih. Lokasi spawn harus tersedia dan valid.
- **Fungsi Objektif**: Menyesuaikan komposisi unit secara dinamis untuk merespons kondisi medan, menjaga keseimbangan antara ekspansi, pertahanan, dan pembersihan wilayah musuh.

3.3.2 Ekspansi dan Pembangunan Tower (Soldier)

Soldier bertugas memperluas wilayah dan membangun tower di atas reruntuhan dengan prioritas tertinggi pada penyelesaian pola tower.

- **Himpunan Kandidat**:
  1. Menyelesaikan/menandai pola tower di reruntuhan terdekat.
  2. Menyerang unit musuh yang terlihat.
  3. Mengecat tile yang belum bercat sekutu.
  4. Bergerak ke suatu arah.
- **Himpunan Solusi**: Satu aksi yang dipilih dan dieksekusi.
- **Fungsi Solusi**: Aksi dipilih secara berurutan berdasarkan prioritas (tower > serang musuh > mengecat > bergerak).
- **Fungsi Seleksi**:
  - **Pembangunan tower**: Pilih reruntuhan terdekat yang belum memiliki tower. Tipe tower ditentukan secara greedy:
    - Jika ada musuh di sekitar ruin $\rightarrow$ *Defense Tower*.

    * Jika *chips* $< 2500 \rightarrow$ *Money Tower*.
    * Selain itu $\rightarrow$ *Paint Tower*.
  - **Pengecatan**: Gunakan *Helper.getGreedyPaintTarget()* yang memberi skor +5 untuk tile musuh dan +6 untuk tile kosong.
  - **Pergerakan**: Hitung skor ekspansi menggunakan *Helper.evaluateFrontierExpansionScore()* dengan komponen:
    * +2 jika ada ruin di lokasi tujuan.
    * +3 jika tile bukan milik sekutu.
    * -2 jika tile sudah milik sekutu.
    * -4 jika ada tower musuh di dekat lokasi.

    Arah dengan skor tertinggi dipilih, dengan bonus +5 jika searah dengan arah eksplorasi.

- **Fungsi Kelayakan**: Pengecekan *canAttack()*, *canMove()*, *canCompleteTowerPattern()*, *canMarkTowerPattern()*, serta ketersediaan sumber daya.
- **Fungsi Objektif**: Memperluas wilayah dan membangun tower untuk memperkuat ekonomi dan pertahanan, dengan prioritas pada area yang memberikan potensi ekspansi terbesar.

3.3.3 Serangan Area Optimal (Splasher)

Splasher memaksimalkan dampak setiap serangan area dengan menargetkan konsentrasi musuh dan area bermusuh.

- **Himpunan Kandidat**: Semua lokasi dalam jangkauan serangan, dan semua arah pergerakan yang mungkin.
- **Himpunan Solusi**: Satu target serangan dan satu arah gerak yang dipilih.
- **Fungsi Solusi**: Target serangan dipilih dan dieksekusi, kemudian pergerakan dipilih.
- **Fungsi Seleksi**:
  - **Serangan**: *Helper.getGreedyAoETarget()* menghitung skor berdasarkan:
    * +20 per unit musuh dalam radius 2 dari pusat serangan.
    * +10 jika tile target berisi cat musuh.
    * +3 jika tile target kosong (netral).

    Target dengan skor tertinggi dipilih.
  - **Pergerakan**: *Helper.evaluateSplasherMovementScore()* menghitung skor berdasarkan:
    * Semakin dekat ke target serangan terpilih, semakin tinggi skor.
    * +5 jika tile tujuan bukan milik sekutu.
    * -10 jika ada tower musuh di dekat tile tujuan.
- **Fungsi Kelayakan**: *canAttack(target)* dan *canMove(dir)*.
- **Fungsi Objektif**: Memaksimalkan kerusakan area dan membersihkan wilayah musuh secara efisien, dengan prioritas pada konsentrasi musuh dan area bermusuh.

3.3.4 Pembersihan dan Dukungan (Mopper)

Mopper bertugas membersihkan cat musuh dan mendukung unit sekutu dengan prioritas bertingkat.

- **Himpunan Kandidat**:
  1. Melakukan *mop swing* ke suatu arah.
  2. Menyerang tile bercat musuh.
  3. Mentransfer cat ke unit sekutu yang kekurangan.
  4. Bergerak ke suatu arah.
- **Himpunan Solusi**: Satu aksi terpilih yang dieksekusi.
- **Fungsi Solusi**: Aksi dipilih secara berurutan sesuai prioritas di atas.
- **Fungsi Seleksi**:
  - **Mop swing**: *Helper.getGreedyMopSwingDir()* memilih arah dengan skor tertinggi (+10 jika ada musuh di tile target).
  - **Serang tile**: *Helper.getGreedyCleanTarget()* memilih tile bercat musuh dengan skor 10.
  - **Transfer cat**: Cari unit *Soldier* atau *Splasher* terdekat yang membutuhkan cat (paintNeeded $\geq$ 50).
  - **Pergerakan**: *Helper.evaluateMopperMovementScore()* menghitung skor berdasarkan:
    * +8 jika tile tujuan berisi cat musuh.
    * +2 jika tile tujuan kosong.
    * +5 jika ada musuh di dekat tile tujuan.
- **Fungsi Kelayakan**: Pengecekan *canMopSwing()*, *canAttack()*, *canTransferPaint()*, dan *canMove()*.
- **Fungsi Objektif**: Membersihkan pengaruh cat musuh dan menjaga keberlangsungan unit sekutu dengan memberikan dukungan logistik tepat waktu.

3.3.5 Navigasi Berbasis Skor (Pergerakan Umum)

Pergerakan semua unit menggunakan pendekatan *greedy scoring* yang mempertimbangkan kondisi lokal.

- **Himpunan Kandidat**: 8 arah mata angin yang mungkin untuk bergerak.
- **Himpunan Solusi**: Satu arah yang dipilih untuk bergerak.
- **Fungsi Solusi**: Solusi ditemukan ketika robot memutuskan untuk bergerak ke suatu arah atau diam.
- **Fungsi Seleksi**: Setiap jenis unit memiliki fungsi skoring spesifik (seperti dijelaskan di atas) yang mengevaluasi potensi keuntungan dari setiap arah pergerakan.

- **Fungsi Kelayakan**: Tujuan pergerakan harus merupakan petak kosong yang dapat dilalui (bukan tembok, bukan robot lain, dan berada dalam peta).
- **Fungsi Objektif**: Mengarahkan robot ke area yang paling memberikan keuntungan sesuai perannya, baik untuk ekspansi, serangan, maupun pembersihan.

3.4 Analisis Efisiensi dan Efektivitas Alternatif Strategi

3.4.1 Alternatif Strategi 1: Money Tower Spam

Analisis Efisiensi Solusi

- **Tower:** Terdapat 3 kandidat produksi unit (Soldier, Splasher, Mopper) dan 1 kandidat upgrade. Pemilihan dilakukan dengan pengecekan kondisi sederhana (jumlah musuh, keberadaan ruin, jumlah chips) sehingga kompleksitasnya O(1).
- **Soldier:** Kandidat aksi meliputi membangun tower, menyerang musuh, mengecat tile, dan bergerak. Prioritas ditentukan secara berurutan. Pengecekan ruin dan tile dilakukan dengan pemindaian linear O(n) terhadap jumlah entitas di sekitar. Kompleksitas dominan adalah O(n) untuk pencarian ruin dan tile.
- **Splasher:** Pemilihan target serangan menggunakan `getBestSplashTarget()` yang menghitung skor setiap tile dalam radius serangan. Untuk setiap kandidat, dilakukan pengecekan 9 tile di sekitarnya. Jika jumlah kandidat adalah m, maka kompleksitasnya O(9m) = O(m). Karena m dibatasi oleh radius serangan, kompleksitasnya konstan.
- **Mopper:** Kandidat aksi meliputi mop swing, serang tile musuh, transfer cat, dan bergerak. Pencarian splasher terdekat yang membutuhkan cat dilakukan dengan pemindaian linear O(n) terhadap jumlah kawan di sekitar. Pencarian arah mop terbaik juga O(8) karena hanya 8 arah.
- **Pergerakan:** Eksplorasi menggunakan vektor tolak-menolak dengan kompleksitas O(n) terhadap jumlah kawan, dilanjutkan pemilihan arah acak O(8).

Kompleksitas waktu algoritma ini secara umum adalah O(n) terhadap jumlah entitas di sekitar dengan faktor konstanta kecil. Karena batasan *bytecode* 17.500 per giliran, strategi ini sangat efisien dan aman dijalankan.

Analisis Efektivitas Solusi

- **Strategi ini efektif apabila:**
  - Peta berukuran besar sehingga ruang ekspansi luas dan waktu untuk membangun ekonomi lebih panjang.
  - Lawan tidak melakukan serangan agresif di awal permainan.
  - Tersedia banyak ruin untuk membangun tower.
- **Strategi ini kurang efektif apabila:**
  - Peta berukuran kecil karena lawan dapat menyerang langsung sebelum ekonomi terbangun.

  - Lawan menerapkan strategi agresif di awal yang dapat menghancurkan tower sebelum sempat berkembang.
  - Ketersediaan ruin terbatas sehingga akumulasi chips terhambat.

3.4.2 Alternatif Strategi 2: Weakest Enemy First

Analisis Efisiensi Solusi

- **Tower:** Pemilihan target serangan unit musuh dengan kompleksitas O(n). Keputusan *spawning* menggunakan operasi *modulo* terhadap `spawnCount`. Pengecekan sumber daya dengan kompleksitas O(1).
- **Soldier:** Kandidat aksi meliputi prioritas penyerangan target HP terendah, penyelesaian pola *ruin*, dan mengecat tile di sekitarnya. Pencarian musuh terlemah punya kompleksitas O(n). Pengecekan *ruin* terdekat juga O(m). Kompleksitas dominannya adalah linear O(n + m) = O(n),
- **Splasher:** Mengecek tile terdekat O(n), mengecek tile musuh juga O(m), sehingga kompleksitasnya O(n + m) = O(n).
- **Mopper:** Mengecek unit sekutu terdekat O(n), mencari unit musuh dengan cat terendah O(m), sehingga kompleksitasnya O(n + m) = O(n).
- **Pergerakan:** Mengecek tile O(n), sementara opsi gerak ada 8 arah O(8).

Secara umum, kompleksitasnya adalah O(n) terhadap jumlah unit musuh terdekat. Karena tidak ada rekursi maupun struktur data dinamis, strategi ini sangat ringan secara komputasi dan aman dari Timeout.

Analisis Efektivitas Solusi

- **Strategi ini efektif apabila:**
  - Peta berukuran kecil.
  - Tower musuh mudah diakses oleh unit sekutu dari tengah.
- **Strategi ini kurang efektif apabila:**
  - Peta berukuran besar.
  - Peta memiliki banyak tembok panjang.

3.4.3 Alternatif Strategi 3: Frontier Expansion Greedy

Analisis Efisiensi Solusi

- **Tower:** Kandidat produksi unit dipilih berdasarkan jumlah musuh dan banyaknya cat musuh di sekitar. Pengecekan dilakukan dengan pemindaian linear O(n) terhadap musuh dan tile di sekitar, namun karena hanya menggunakan informasi lokal, kompleksitasnya O(n) dengan n kecil.

- **Soldier:** Kandidat aksi meliputi pembangunan tower, serangan musuh, pengecatan tile, dan pergerakan. Prioritas ditentukan secara berurutan. Untuk pergerakan, dilakukan evaluasi skor frontier pada setiap arah (8 arah) dengan fungsi `evaluateFrontierExpansionScore()` yang memeriksa keberadaan ruin, status tile, dan ancaman tower musuh. Kompleksitasnya O(8) = konstan. Pencarian ruin dan tile juga O(n) dengan n jumlah entitas di sekitar.
- **Splasher:** Pemilihan target serangan menggunakan `getGreedyAoETarget()` yang menghitung skor setiap tile dalam radius serangan. Untuk setiap tile, dihitung jumlah musuh dalam radius 2 (maksimal 4 tile) dan status tile. Jika jumlah kandidat adalah m, kompleksitasnya O(4m) = O(m). Pergerakan menggunakan `evaluateSplasherMovementScore()` yang mengevaluasi 8 arah dengan O(8) per arah (pengecekan jarak dan tile), sehingga total O(8×8) = O(64) konstan.
- **Mopper:** Kandidat aksi meliputi mop swing, serang tile musuh, transfer cat, dan pergerakan. `getGreedyMopSwingDir()` mengecek 8 arah dengan O(8). `getGreedyCleanTarget()` memindai tile di sekitar O(n). Pergerakan menggunakan `evaluateMopperMovementScore()` yang mengecek 8 arah dengan O(8) per arah. Kompleksitas dominan O(n) untuk pencarian tile musuh.

Kompleksitas waktu algoritma secara umum adalah O(n) terhadap jumlah entitas di sekitar, dengan faktor konstanta kecil. Semua fungsi skoring dirancang efisien dan aman dalam batasan *bytecode*.

Analisis Efektivitas Solusi

- **Strategi ini efektif apabila:**
  - Peta memiliki banyak area kosong atau netral yang perlu diekspansi.
  - Musuh terkonsentrasi di area tertentu sehingga *Splasher* dapat memberikan damage maksimal.
  - Banyak cat musuh yang perlu dibersihkan sehingga *Mopper* dapat berperan optimal.
  - Tersedia ruin di area frontier sehingga *Soldier* dapat membangun tower untuk memperkuat pertahanan.
- **Strategi ini kurang efektif apabila:**
  - Peta sempit dengan sedikit area ekspansi sehingga skor frontier tidak bervariasi.
  - Musuh tersebar merata sehingga serangan area *Splasher* kurang optimal.
  - Koordinasi antarunit buruk yang menyebabkan beberapa unit berebut area yang sama.
  - Tidak ada mekanisme komunikasi sehingga informasi frontier tidak tersebar dan ekspansi menjadi tidak efisien.

3.5 Strategi Greedy yang Dipilih

Strategi yang dipilih adalah *Money Tower Spam* karena menang lebih banyak ketika diadu dengan *Weakest Enemy First* maupun *Frontier Expansion Greedy*.

4 Implementasi dan Pengujian

4.1 Implementasi Tiga Alternatif Solusi Greedy

Berikut adalah pseudocode untuk tiga alternatif strategi greedy yang dirancang. Setiap strategi memiliki heuristik yang berbeda sesuai dengan tujuan optimasi masing-masing.

4.1.1 Alternatif 1: Money Tower Spam

```
repeat

    if robot_adalah_tower? = ya then

        if bisa_upgrade_tower? = ya then
            if chips_banyak? = ya then
                Upgrade_Tower

        Cari_Ruin_Di_Sekitar
        Cari_Lokasi_Spawn

        if lokasi_spawn_tersedia? = ya then

            if ada_ruin_kosong? = ya then
                Buat_Robot_Soldier

            else
                if ronde_awal? = ya then
                    Buat_Robot_Soldier

                else
                    if ekonomi_sudah_besar? = ya then
                        Buat_Robot_Splasher

                    else
                        if ada_musuh_dekat? = ya then
                            Buat_Robot_Mopper

        { Pada titik ini tower sudah mencoba membuat robot sesuai
        kondisi permainan }


    else if robot_adalah_soldier? = ya then

        Cari_Ruin_Di_Sekitar

        if ronde_awal? = ya then
            Hitung_Jumlah_Tile_Dicat_Tim
```

```
            if area_cat_masih_sedikit? = ya then
                Cari_Lokasi_Resource_Pattern

                if lokasi_resource_ditemukan? = ya then
                    if robot_sudah_di_lokasi? = ya then
                        Tandai_Resource_Pattern
                    else
                        Bergerak_Ke_Lokasi

        { Pada titik ini soldier mencoba membuat resource pattern }

        if ada_ruin? = ya then
            if pola_tower_selesai? = ya then
                Bangun_Tower

        if resource_pattern_selesai? = ya then
            Selesaikan_Resource_Pattern

        Cari_Tile_Yang_Belum_Dicat

        if tile_ditemukan? = ya then
            if dalam_radius_serang? = ya then
                Cat_Tile
            else
                Bergerak_Ke_Tile

        else
            if ada_ruin_kosong? = ya then
                Tandai_Pola_Tower
                Bergerak_Ke_Ruin
            else
                Eksplorasi_Map


    else if robot_adalah_splasher? = ya then

        if paint_habis? = ya then
            Kembali_Ke_Tower
            { Di titik ini splasher akan mengisi ulang paint }

        else
            Cari_Target_Splash

            if target_ditemukan? = ya then
                Lakukan_Splash_Attack
```

```
            Bergerak_Ke_Area_Belum_Dicat


     else {robot_adalah_mopper? = ya}

        if paint_sedikit? = ya then
            Cari_Tower_Paint_Terdekat
            Bergerak_Ke_Tower

        else
            if ada_musuh_dekat? = ya then
                Serang_Musuh
                Bergerak_Ke_Musuh

            else
                Cari_Splasher_Yang_Butuh_Paint

                if splasher_ditemukan? = ya then
                    Transfer_Paint
                    Bergerak_Ke_Splasher
                else
                    Eksplorasi_Map

until (permainan_selesai? = ya)
```

4.1.2 Alternatif 2: Weakest Enemy First

```
repeat

    if robot_adalah_tower? = ya then

        if ada_musuh_dalam_jangkauan? = ya then
            Serang_Musuh_HP_Terendah_(WEF)

        Kirim_Dan_Terima_Pesan_Simetri

        if bisa_upgrade_tower? = ya then
            if chips_lebih_dari_3000? = ya then
                Upgrade_Self
                { Tower melakukan self-upgrade dan menunda spawn }

        Hitung_Antrean_Spawn_(Modulo_4)

        if slot_0_atau_1? = ya then
            Buat_Robot_Soldier
```

```
        else if slot_2? = ya then
            Buat_Robot_Mopper
        else { slot_3 }
            if chips_lebih_dari_500? = ya then
                Buat_Robot_Splasher
            else
                Buat_Robot_Mopper


    else if robot_adalah_soldier? = ya then

        if mode_refill_aktif? = ya then
            Antre_Ke_Paint_Tower
            { Menghentikan aksi lain hingga cat terisi }

        Cari_Tower_Musuh_Terlemah
        if tower_musuh_ditemukan? = ya then
            Serang_Tower_Musuh

        Cari_Ruin_Di_Sekitar
        if ada_ruin_dengan_pola_belum_selesai? = ya then
            Cat_Pola_Tower_Pada_Ruin

        Cek_Radar_Markas_Musuh
        if target_markas_terbukti_kosong? = ya then
            Ganti_Target_Simetri_Instan

        Bergerak_Greedy_(Prioritas_Cat_Sekutu)

        if action_ready? = ya then
            Cat_Ubin_Kosong_Di_Bawah_Kaki_(Path_Paving)


    else if robot_adalah_splasher? = ya then

        if mode_refill_aktif? = ya then
            Antre_Ke_Paint_Tower

        if action_ready? = ya then
            if ada_tower_musuh_sekarat? = ya then
                Serang_Tower
            else
                Kalkulasi_Skor_Area_(Greedy_Coverage)
                if skor_area_lebih_dari_4? = ya then
                    Lakukan_Splash_Attack

        Cek_Radar_Markas_Musuh
```

```
        if target_markas_terbukti_kosong? = ya then
            Ganti_Target_Simetri_Instan

        Cari_Gumpalan_Cat_Musuh_Terdekat
        if gumpalan_cat_ditemukan? = ya then
            Bergerak_Mendekati_Cat_Musuh_(Paint_Seeking)
        else
            Bergerak_Greedy_Ke_Markas_Musuh


    else { robot_adalah_mopper? = ya }

        if ada_sekutu_kritis_(cat < 10)? = ya then
            Transfer_Cat_Ke_Sekutu_(Combat_Medic)

        if musuh_berkerumun_di_sebelah? = ya then
            Lakukan_Mop_Swing_(Lifesteal)
        else if ada_musuh_tunggal? = ya then
            Serang_Musuh_Dengan_Cat_Terendah

        Cari_Tile_Cat_Musuh
        if tile_ditemukan? = ya then
            Hapus_Cat_Musuh_(Vanguard)

        Cari_Ruin_Kosong
        if ruin_ditemukan? = ya then
            Bangun_Tower_Di_Ruin

        Cari_Soldier_Terdekat
        if soldier_ditemukan? = ya then
            Ikuti_Pergerakan_Soldier
        else
            Cek_Radar_Markas_Musuh
            if target_markas_terbukti_kosong? = ya then
                Ganti_Target_Simetri_Instan
            Bergerak_Greedy_Ke_Markas_Musuh

until (permainan_selesai? = ya)
```

4.1.3 Alternatif 3: Frontier Expansion Greedy

```
repeat

    if robot_adalah_tower? = ya then
        Hitung_Jumlah_Musuh_di_Sekitar
```

```
        if jumlah_musuh_banyak? = ya then
            Produksi_Splasher
        else if banyak_cat_musuh? = ya then
            Produksi_Mopper
        else
            Produksi_Soldier
        Cek_Sumber_Daya
        if sumber_daya_cukup? = ya then
            Cari_Lokasi_Spawn_Terbaik
            if lokasi_spawn_ditemukan? = ya then
                Produksi_Robot
        Serang_Musuh_Jika_Ada

    else if robot_adalah_soldier? = ya then
        Cek_Aksi
        if bisa_melakukan_aksi? = ya then
            Cari_Ruin_Di_Sekitar
            if ada_ruin_kosong? = ya then
                Tentukan_Tipe_Tower_Berdasarkan_Kondisi
                if pola_tower_selesai? = ya then
                    Bangun_Tower
                else if bisa_menandai_tower? = ya then
                    Tandai_Pola_Tower
                else
                    Cari_Tile_Blueprint_Yang_Perlu_Dicat
                    if tile_blueprint_ditemukan? = ya then
                        Cat_Tile_Sesuai_Mark
            if belum_melakukan_aksi? = ya then
                Cari_Target_Musuh_Terbaik
                if target_musuh_ditemukan? = ya then
                    Serang_Musuh
            if belum_melakukan_aksi? = ya then
                Cari_Tile_Terbaik_Untuk_Dicat
                if tile_target_ditemukan? = ya then
                    Cat_Tile
        Cek_Pergerakan
        if bisa_bergerak? = ya then
            if sedang_membangun_ruin? = ya then
                if jarak_ke_ruin_jauh? = ya then
                    Gerak_Menuju_Ruin
                else
                    Gerak_Acak
            else
                Hitung_Skor_Frontier_Untuk_Setiap_Arah
                Pilih_Arah_Dengan_Skor_Tertinggi
                if arah_terbaik_ditemukan? = ya then
                    Gerak_Ke_Arah_Tersebut
```

```
                else
                    Ganti_Arah_Eksplorasi

    else if robot_adalah_splasher? = ya then
        Cek_Aksi
        if bisa_melakukan_aksi? = ya then
            Cari_Target_Serangan_Area_Terbaik
            if target_serangan_ditemukan? = ya then
                Lakukan_Serangan_Area
        Cek_Pergerakan
        if bisa_bergerak? = ya then
            Hitung_Skor_Pergerakan_Untuk_Setiap_Arah
            Pilih_Arah_Dengan_Skor_Tertinggi
            if arah_terbaik_ditemukan? = ya then
                Gerak_Ke_Arah_Tersebut

    else {robot_adalah_mopper? = ya}
        Cek_Aksi
        if bisa_melakukan_aksi? = ya then
            Cari_Arah_Mop_Terbaik
            if arah_mop_ditemukan? = ya then
                Lakukan_Mop_Swing
            if belum_melakukan_aksi? = ya then
                Cari_Tile_Cat_Musuh_Terdekat
                if tile_musuh_ditemukan? = ya then
                    Serang_Tile_Tersebut
        if belum_melakukan_aksi? = ya and bisa_melakukan_aksi? = ya and
                paint_cukup? = ya then
            Cari_Kawan_Yang_Butuh_Cat
            if kawan_ditemukan? = ya then
                Transfer_Cat
        Cek_Pergerakan
        if bisa_bergerak? = ya then
            Hitung_Skor_Pergerakan_Berdasarkan_Cat_Musuh
            Pilih_Arah_Dengan_Skor_Tertinggi
            if arah_terbaik_ditemukan? = ya then
                Gerak_Ke_Arah_Tersebut

until (permainan_selesai? = ya)
```

4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur pada Bot Utama

4.2.1 Struktur Data

Implementasi unit dirancangan tanpa struktur data kompleks supaya menghemat komputasi. Struktur data utama meliputi:

- Array Primitif:
  1. *RobotInfo[] menyimpan himpunan unit robot sekutu maupun musuh untuk evaluasi target.
  2. MapLocation[] menyimpan koordinat seperti ruin dan tile target.
  3. MapInfo[] menyimpan informasi tile seperti jenis cat dan keberdaan pola.
  4. Direction[] menyimpan 8 arah untuk pilihan gerakan
- Memori Statik: *currentExploreDir dan currentSplasherDir mencegah unit bergerak bolak-balik saat mengeksplorasi peta.

4.2.2 Prosedur dan Fungsi Utama

Source code dibagi menjadi dua kelas utama, yaitu RobotPlayer.java dan Helper.java

- Kelas RobotPlayer.java
  1. runTower(RobotController rc): mengatur logika menara. Fungsi ini memiliki insting *defensif* (menyerang musuh dengan HP terendah) dan *reproduktif*. Spawning diatur berdasarkan kondisi ekonomi: mencetak Soldier untuk ekspansi awal, mencetak Mopper jika ada ancaman musuh dekat, dan melakukan *spam* splasher ketika ekonomi meledak (currentChips ¿ 3000) atau masuk fase *late game* (currentRound ¿ 1250—). Menara juga akan memicu *Self-Upgrade* jika dana melebihi 5000 *chips*.
  2. runSoldier(RobotController rc): menentukan variabel towerToBuild. Jika game masih berada di bawah ronde 125, Soldier dipaksa untuk mendedikasikan seluruh ruin yang ditemuinya menjadi LEVEL_ONE_MONEY_TOWER. Soldier juga bertugas mencari Resource Pattern (SRP) di awal permainan dan melakukan pengecatan.
  3. runSplasher(RobotController rc): fokus pada dua hal, yaitu menembakkan cat ke area dengan kepadatan musuh tertinggi (skor maksimal) dan berjalan mencari area yang belum diwarnai (moveTowardsUnpaintedArea).
  4. runMopper(rc): Mopper akan menghisap cat musuh terdekat (Helper.getClosestRobot) dan memberi cat (transferPaint) kepada Splasher sekutu yang kehabisan cat.
  5. moveTowardsUnpaintedArea(RobotController rc): navigasi khusus Splasher yang menggunakan kalkulasi skor untuk menjauhi Splasher sekutu lain dan memprioritaskan area musuh atau area kosong.
- Kelas Helper.java
  1. getBestSplashTarget(): mengecek radius serangan, menghitung jumlah tile musuh atau netral, dan memaksimal AeO Splasher.
  2. moveToLoc() / retreatForRefill(): memandu unit menuju koordinat tertentu atau mengisi ulang cat di Paint Tower terdekat.
  3. chooseNewExplorationDirection(): menjumlahkan vektor koordinat sekutu untuk menghindari kerumunan sehingga unit bisa menyebar lebih luas.

4.2.3 Penerapan Algoritma Greedy

- Greedy Target Selection pada Splasher (Helper.getBestSplashTarget): memilih koordinat pusat jatuhnya cat yang mengenai jumlah ubin non-sekutu (musuh/kosong) paling banyak (validPaintCount ¿= 4). Keputusan tembakan diambil saat itu juga demi merebut wilayah maksimal.
- Greedy Attack pada Tower (runTower): mencari nilai minimum (HP terendah) melalui variabel lowestHP. Menara secara sepihak akan langsung menembak musuh dengan HP terendah.
- Greedy Navigation pada Soldier dan Mopper (Helper.moveToLoc): hanya melihat satu petak di depannya. Jika arah lurus bisa dilewati, unit akan langsung maju. Jika terhalang, unit mengambil alternatif serong terdekat.

4.3 Pengujian Greedy

Pada bagian ini, dilakukan pengujian dengan mempertandingkan bot utama (Main Bot) melawan bot alternatif pada berbagai peta. Setiap pengujian dilakukan sebanyak satu kali pertandingan hingga ronde 2000 atau hingga ada pemenang. Berikut adalah hasil pengujian.

4.3.1 Pengujian 1: Main Bot vs Alt 1 (Money Tower Spam) pada Peta Besar

Konfigurasi pertandingan:

- Robot Controller: Java
- Tim A: main_bot
- Tim B: alternative_bots_1
- Peta: headphones

```
[server] -------------------- Match Starting --------------------
[server] main_bot vs. alternative_bots_1 on headphones
[server] main_bot (A) wins (round 2000)
[server] Reason: The winning team won on tiebreakers (painted more of the map).
[server] -------------------- Match Finished --------------------
Closed: /[0:0:0:0:0:0:0:1]:62286 for
```

Dari hasil tersebut, terlihat bahwa Main Bot menang melalui tiebreaker (area lebih luas). Hal ini menunjukkan bahwa strategi Main Bot lebih efektif dalam memperluas wilayah dalam jangka panjang.

4.3.2 Pengujian 2: Main Bot vs Alt 1 pada Peta dengan Banyak Tembok

Konfigurasi pertandingan:

- Robot Controller: Java
- Tim A: main_bot
- Tim B: alternative_bots_1
- Peta: yearofthesnack

```
[server] main_bot (A) wins (round 2000)
[server] Reason: The winning team won on tiebreakers (painted more of the map).
[server] -------------------- Match Finished --------------------
Closed: /127.0.0.1:63974 for
```

Dari hasil tersebut, terlihat bahwa Main Bot menang melalui tiebreaker (area lebih luas). Hal ini menunjukkan bahwa strategi Main Bot tetap efektif meskipun pada peta dengan banyak tembok.

4.3.3 Pengujian 3: Main Bot vs Alt 1 pada Peta Kecil

Konfigurasi pertandingan:

- Robot Controller: Java
- Tim A: main_bot

- Tim B: alternative_bots_1
- Peta: DefaultSmall

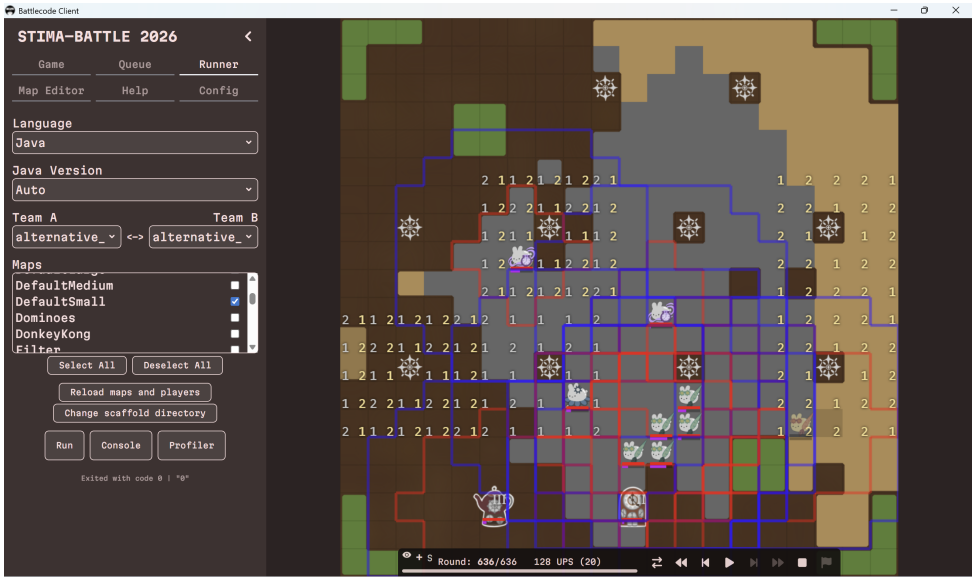
```
[server] -------------------- Match Starting --------------------
[server] main_bot vs. alternative_bots_1 on DefaultSmall
[server] alternative_bots_1 (B) wins (round 596)
[server] Reason: The winning team destroyed all of the enemy team's units.
[server] -------------------- Match Finished --------------------
Closed: /127.0.0.1:56061 for
```

Dari hasil tersebut, terlihat bahwa Alternatif 1 (Money Tower Spam) berhasil memenangkan pertandingan pada ronde 596 dengan menghancurkan seluruh unit lawan. Hal ini menunjukkan bahwa pada peta kecil, strategi Money Tower Spam yang agresif dapat mengalahkan Main Bot karena ruang gerak terbatas dan akses ke ruin lebih cepat.

4.3.4 Pengujian 4: Alt 1 vs Alt 2 pada Peta Kecil

Konfigurasi pertandingan:

- Robot Controller: Java
- Tim A: alternative_bots_1
- Tim B: alternative_bots_2
- Peta: DefaultSmall




```
[server] alternative_bots_1 (A) wins (round 635)
[server] Reason: The winning team destroyed all of the enemy team's units.
[server] -------------------- Match Finished --------------------
Closed: /127.0.0.1:58229 for

BUILD SUCCESSFUL in 5s
6 actionable tasks: 2 executed, 4 up-to-date
Exited with code 0 | "0"
```

Dari hasil tersebut, terlihat bahwa Alternatif 1 (Money Tower Spam) berhasil memenangkan pertandingan pada ronde 635 dengan menghancurkan seluruh unit lawan. Hal ini menunjukkan bahwa pada peta yang ukurannya kecil, strategi ekonomi yang agresif lebih unggul.

4.4 Analisis Hasil Pengujian

Dengan strategi utama, *Money Tower Spam*, game berhasil dimenangkan lewat ekspansi wilayah. Strategi greedy berhasil mencapai tujuan dengan memilih pembanguunan yang paling sesuai dengan keadaan chips saat itu. Selain itu, navigasi dan perilaku setiap unit juga ditentukan secara greedy berdasarkan tile di sekitarnya. Strategi ini murah secara waktu dan komputasi sehingga tidak menyebabkan timeout. *Money Tower Spam* paling unggul ketika punya waktu untuk mengembangkan ekonomi pembangunannya, misalnya karena peta yang luas atau tower awal yang tertutup oleh tembok-tembok. Namun, strategi ini lemah terhadap serangan agresif di awal karena pembangunan belum cukup tersebar.

5 Kesimpulan dan Saran

5.1 Kesimpulan

Dengan strategi greedy, tujuan memenangkan Battlecode bisa tercapai, baik dengan ekspansi wilayah maupun menghancurkan semua unit musuh. Strategi utama, *Money Tower Spam*, punya kelebihan menavigasikan peta yang luas dan kompleks dengan banyak tembok, tetapi rentan terhadap serangan agresif di awal permainan ketika ekonomi belum

terbangun. Strategi *Weakest Enemy First* unggul dalam peta kecil dengan sedikit tembok karena langsung menyerang tower dan unit musuh terlemah sejak awal game. Namun, strategi ini kurang baik dalam menavigasikan peta yang luas, banyak tembok, serta dinamika *late game* ketika musuh sudah membangun wilayah dan banyak tower. Terakhir, Frontier Expansion Greedy cocok dengan peta yang punya banyak area kosong dan musuh yang terkonsentrasi di satu area. Namun, strategi ini kurang cocok menavigasikan peta dengan sedikit area ekspansi.

5.2 Saran

Saran untuk pengembangan strategi adalah mengoptimalkan komunikasi, SRP, dan defense tower pada *late game* ketika ekonomi dan sumber daya sudah cukup stabil. Namun, pergerakan musuh di awal juga perlu diperhatikan, terutama perlindungan tower pada awal game yang rentan diserang langsung. Dengan begitu, sebaiknya parameter strategi bersifat dinamis tergantung ekspansi wilayah dan sumber daya yang sedang berjalan, bentuk peta, dan pergerakan musuh.

Lampiran

Tautan Repository GitHub

Kode sumber bot ini dapat diakses pada repository GitHub berikut:
`https://github.com/mhps-null/Tubes1_Nayakuy`

Tautan Video

Video penjelasan dan simulasi bot dapat dilihat pada tautan berikut:
`https://youtu.be/-i4CV370ivc`

Daftar Pustaka

References

[1] Battlecode, "Battlecode 2025 Specification," 2025. [Daring]. Tersedia: `https://releases.battlecode.org/specs/battlecode25/3.1.0/specs.pdf`

[2] Gradle, "Installing Gradle," 2026. [Daring]. Tersedia: `https://docs.gradle.org/current/userguide/installation.html`

[3] Munir, R., "Algoritma Greedy (Bagian 1)," Institut Teknologi Bandung, Feb. 2026. [Daring]. Tersedia: `https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-(2026)-Bag1.pdf`

[4] Munir, R., "Algoritma Greedy (Bagian 2)," Institut Teknologi Bandung, Feb. 2026. [Daring]. Tersedia: `https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/05-Algoritma-Greedy-(2026)-Bag2.pdf`

[5] Munir, R., "Algoritma Greedy (Bagian 3)," Institut Teknologi Bandung, Feb. 2026. [Daring]. Tersedia: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/06-Algoritma-Greedy-(2026)-Bag3.pdf

Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Muhammad Haris Putra Sulastianto
NIM: 13524053

Kalyca Nathania B. Manullang
NIM: 13524071

Vara Azzara Ramli Pulukadang
NIM: 13524091